Low-Level Design (LLD)

🔹 Objective

The objective of the Low-Level Design (LLD) is to define the internal implementation details of the RAG-based Customer Support Assistant system.

It provides a detailed view of how each component is designed, structured, and interacts at the code level to process user queries and generate responses.

The LLD focuses on:

- Breaking down the system into individual modules such as document processing, chunking, embedding generation, retrieval, and response generation
- Defining the internal data structures used to represent documents, chunks, embeddings, and responses
- Explaining the workflow execution using LangGraph, including nodes, edges, and state transitions
- Detailing the conditional routing logic for decision-making
- Describing the implementation of the Human-in-the-Loop (HITL) mechanism
- Outlining API interfaces and error handling strategies

This design ensures that the system is modular, maintainable, and easy to extend or modify in future enhancements.

The design closely aligns with the implemented code structure, ensuring consistency between design and execution.

1. Module-Level Design

🔹 Overview

The system is divided into multiple modules, each responsible for a specific functionality. This modular design improves maintainability, scalability, and ease of debugging.

🔹 1. Document Processing Module

This module is responsible for loading and preparing the input document.

Functions:

- Load PDF file

- Extract raw text
- Prepare data for chunking

Implementation: loader.py

🔹 2. Chunking Module

This module splits the extracted text into smaller chunks for efficient processing.

Functions:

- Divide text into manageable segments
- Maintain context using overlap
- Optimize chunk size for embedding

Configuration:

- Chunk size: ~500
- Overlap: ~50

Implementation: chunker.py

🔹 3. Embedding Module

This module converts text chunks into vector representations.

Functions:

- Generate embeddings for each chunk
- Capture semantic meaning of text
- Prepare data for vector storage

Implementation: embeddings.py

🔹 4. Vector Storage Module

This module stores embeddings in a vector database.

Functions:

- Store embeddings with metadata
- Enable fast similarity search
- Maintain persistent storage

Technology Used: ChromaDB

🔹 5. Retrieval Module

This module retrieves relevant chunks based on user queries.

Functions:

- Convert query into embedding
- Perform similarity search
- Return top relevant chunks

Implementation: retriever.py

🔹 6. Query Processing Module

This module handles user queries and prepares them for processing.

Functions:

- Accept user input
- Validate query
- Forward query to workflow engine

🔹 7. Graph Execution Module (LangGraph)

This module controls the execution flow using a graph-based approach.

Functions:

- Define nodes and edges
- Manage state transitions
- Execute processing steps in sequence

Implementation: graph.py

🔹 8. HITL Module (Human-in-the-Loop)

This module handles escalation to human support.

Functions:

- Detect low-confidence responses
- Forward query to human
- Receive and return human response

2. Data Structures

🔹 Overview

The system uses structured data formats to represent documents, chunks, embeddings, queries, and workflow states. These structures ensure consistency and smooth data flow across modules.

🔹 1. Document Representation

The document is represented as raw extracted text along with optional metadata.

```
Document = {
  "text": "Full extracted text from PDF",
  "metadata": {
    "source": "file_name.pdf"
  }
}
```

🔹 2. Chunk Format

The document is divided into smaller chunks for processing.

```
Chunk = {
  "id": "chunk_1",
  "content": "Text content of the chunk",
  "metadata": {
    "position": 1
  }
}
```

🔹 3. Embedding Structure

Each chunk is converted into a numerical vector representation.

```
Embedding = {
  "chunk_id": "chunk_1",
  "vector": [0.12, 0.45, 0.78, ...],
}
```

4. Query-Response Schema

This structure represents user input and system output.

```
QueryResponse = {
  "query": "What is the refund policy?",
  "retrieved_chunks": ["chunk_3", "chunk_7"],
  "answer": "The refund policy allows...",
  "confidence": 0.82
}
```

5. State Object for Graph (LangGraph)

The state object is used to pass data between nodes in the workflow.

```
State = {
  "query": "...",
  "retrieved_data": "...",
  "response": "...",
  "confidence": 0.0,
  "escalation": False
}
```

These data structures enable efficient data handling and seamless communication between different modules of the system.

3. Workflow Design (LangGraph)

Overview

The system uses LangGraph to implement a graph-based workflow for processing user queries. This approach allows flexible control over execution flow, state management, and decision-making.

The workflow is designed as a series of nodes connected by edges, where data flows through a shared state object.

🔹 1. Nodes

🔸 Processing Node

This node performs the core operations of the system.

Functions:

- Receive user query from state
- Convert query into embedding
- Retrieve relevant chunks from vector database
- Pass retrieved context to the LLM
- Generate response

🔸 Output Node

This node handles the final stage of the workflow.

Functions:

- Evaluate response quality
- Check confidence score
- Decide whether to return response or escalate
- Send final output to the user

🔹 2. Edges (Flow Between Nodes)

Edges define how execution moves from one node to another.

Workflow Flow:

Start → Processing Node → Output Node → End

- The query first enters the **Processing Node**
- After processing, control moves to the **Output Node**
- Based on decision logic, the workflow ends or triggers escalation

🔹 3. State (Data Flowing Between Nodes)

The system uses a shared state object to pass data between nodes.

State includes:

- User query
- Retrieved chunks
- Generated response
- Confidence score
- Escalation flag

Example:

```
State = {
  "query": "What is refund policy?",
  "retrieved_data": "...",
  "response": "...",
  "confidence": 0.75,
  "escalation": False
}
```

🔹 4. Execution Flow

1. The workflow starts with the user query
2. The query is processed in the Processing Node
3. Relevant data is retrieved and response is generated
4. The Output Node evaluates the response
5. Based on confidence, the system either:
    - Returns the answer
    - Triggers HITL escalation

The graph-based workflow enables flexible control, making it easy to extend the system with additional nodes and decision paths.

4. Conditional Routing Logic

🔹 Overview

The system uses conditional routing logic to decide whether a generated response should be returned to the user or escalated to a human agent. This decision is based on response quality, confidence level, and availability of relevant context.

🔹 1. Answer Generation Criteria

A response is directly returned to the user when:

- Relevant chunks are successfully retrieved from the vector database
- The LLM generates a meaningful and context-aware answer
- The response aligns with the retrieved document content
- The confidence score is above a defined threshold (e.g., > 0.6)

In such cases, the system considers the response reliable and sends it to the user.

🔹 2. Escalation Criteria

The query is escalated to the HITL module when:

- The system is unable to retrieve relevant information
- The generated response is incomplete or unclear
- The confidence score is below the threshold
- The query requires human judgment or domain expertise

🔹 3. Low Confidence Handling

Low confidence is determined based on:

- Weak similarity scores from retrieval
- Poor alignment between query and retrieved chunks
- Uncertain or vague LLM responses

When confidence is low:

- The system avoids giving incorrect answers
- The query is forwarded to human support

🔹 4. Missing Context

If no relevant chunks are retrieved:

- The system identifies a lack of supporting data
- It avoids generating hallucinated responses
- The query is directly escalated to the HITL module

🔹 5. Complex Query Handling

Some queries may be too complex for automated processing.

Examples:

- Multi-step reasoning questions
- Ambiguous or unclear queries
- Queries outside the document scope

In such cases:

- The system triggers escalation
- A human agent provides the final response

This conditional routing mechanism ensures reliability by prioritizing accuracy over automation.

5. HITL Design (Human-in-the-Loop)

🔹 Overview

The Human-in-the-Loop (HITL) module ensures reliability by involving human intervention when the system is unable to generate a confident or accurate response.

🔹 1. When Escalation is Triggered

Escalation to human support is triggered under the following conditions:

- The confidence score of the generated response is below a predefined threshold
- No relevant chunks are retrieved from the vector database
- The query is complex, ambiguous, or outside the document scope

- The generated response is incomplete or not meaningful

🔹 2. What Happens After Escalation

Once escalation is triggered:

- The user query is forwarded to a human agent or support system
- The system temporarily pauses automated response generation
- The human agent reviews the query and prepares an appropriate response

🔹 3. Integration of Human Response

After the human agent provides a response:

- The response is captured by the system
- It is formatted and sent back to the user as the final answer
- The system may optionally log the response for future improvements

HITL enhances system reliability by ensuring that critical or uncertain queries are handled accurately by human expertise.

6. API / Interface Design

🔹 Overview

The system provides a simple interface for users to interact with the RAG-based assistant. It supports query input and returns structured responses, ensuring smooth communication between the user and the system.

🔹 1. Input Format

The system accepts user queries in a structured format.

Example:

```
{
  "query": "What is the refund policy?"
}
```

Description:

- query: The question asked by the user in natural language

🔹 2. Output Format

The system returns a structured response containing the generated answer and related details.

Example:

```
{
  "answer": "The refund policy allows returns within 7 days.",
  "confidence": 0.82,
  "source": "Document chunk reference"
}
```

Description:

- answer: Generated response from the LLM
- confidence: Score indicating reliability
- source: Reference to retrieved document content

🔹 3. Interaction Flow

The interaction between the user and the system follows these steps:

1. The user submits a query through the interface (CLI/Web)
2. The query is validated and passed to the processing pipeline
3. The system retrieves relevant data from the vector database
4. The LLM generates a response using retrieved context
5. The routing logic evaluates the response
6. The final answer is returned to the user or escalated to HITL

The interface is designed to be simple, consistent, and easily extendable for web or API-based integrations.

7. Error Handling

🔹 Overview

The system includes error handling mechanisms to ensure stability and reliability during failures. It manages different types of errors that may occur during document processing, retrieval, and response generation.

🔹 1. Missing Data

This occurs when:

- The PDF document is not loaded properly
- The document contains insufficient or empty content

Handling Strategy:

- Validate document before processing
- Display appropriate error message to the user
- Prevent further execution until valid data is available

🔹 2. No Relevant Chunks Found

This occurs when:

- The retriever fails to find matching content in the vector database

Handling Strategy:

- Avoid generating unsupported or hallucinated responses
- Trigger escalation to the HITL module
- Inform the user that no relevant information was found

🔹 3. LLM Failure

This occurs when:

- The LLM API fails to respond
- Timeout or network issues occur

Handling Strategy:

- Retry the request (if applicable)
- Provide fallback response message

- Log the error for debugging
- Optionally escalate to HITL

Robust error handling ensures system reliability and prevents incorrect or misleading responses.